Data Wrangling 🤠

For Loops

Lesson 10

The While Loop Pattern

- Processing every element in a list with a `while` loop always looks the same:

```
temps = [0.0, 20.0, 37.0]
i = 0

while i < len(temps):
    item = temps[i]    # access element
    # process item
    i = i + 1          # increment
```

`for` Loops

- A simpler way to process every element in a sequence

```
temps = [0.0, 20.0, 37.0]

for temp in temps:
    # process temp
```

- Python handles the indexing and incrementing for you
- `temp` is automatically assigned to each element, one at a time

`for` Syntax

```
for [identifier] in [sequence]:
    [repeat block]
    # identifier holds the current element
```

- `identifier` — a variable name you choose for each element
- `sequence` — any list, string, or range
- Loop ends automatically after the last element

Demo: `for` vs. `while`

```python
# for.py
temps = [0.0, 20.0, 37.0]

# while loop:
i = 0
while i < len(temps):
    print(temps[i])
    i = i + 1

# for loop:
for temp in temps:
    print(temp)
```

Demo: Converting Temperatures

```python
# for.py
temps = [0.0, 20.0, 37.0]
converted = []

for temp in temps:
    result = celsius_to_fahrenheit(temp)
    converted.append(result)

print(converted)
```

Demo: `for` Loops and Mutation

- Modifying the `for` loop variable does *not* update the list
- *Must* use access by index to change list values

```python
# for_continued.py
names = ["Vrinda", "Alicia", "Taylor"]
for name in names:
    name = name + "!"
# names is still ["Vrinda", "Alicia", "Taylor"]
i = 0
while i < len(names):
    names[i] = names[i] + "!"
    i = i + 1
# names is updated to ["Vrinda!", "Alicia!", "Taylor!"]
```

Demo: Other Sequences

- We can write for loops over `str` objects

```
# for_continued.py
for char in "hello":
    print(char) # will output every char one by one
```

When to Use `for` vs. `while`

	`for`	`while`
Want to process every element	✅	✅
Need to know the index	❌	✅
Stop early based on a condition	❌	✅
Count up/down by custom step	❌	✅